Lowering the Bar

How Large Language Models Can be Used as a Copilot by Hardware Hackers

Jason Blocklove | New York University Tandon School of Engineering
Hammond Pearce | University of New South Wales
Ramesh Karri | New York University Tandon School of Engineering

Large language models (LLMs) can write code in "hardware languages" defining circuits. To learn if LLMs can be asked to add vulnerabilities to hardware, we ran a global competition. We found that teams could introduce even severe vulnerabilities, implying insufficient LLM guardrails.

Large Language Models (LLMs), a type of transformer-based artificial intelligence (AI) model, have for years demonstrated capabilities for a wide variety of tasks related to the development, debugging, and modification of code. These LLMs can either be specifically trained to complete simple tasks autonomously or can otherwise be used in conjunction with human engineers to build more complex designs. Such LLM assistance can also be useful for tasks in hardware design, as modern integrated circuits and computer chips tend to being their lives as blocks of "hardware description language" (HDL) code such as Verilog and VHDL. As an example, in our prior work "Chip-Chat"[1] we showed how ChatGPT—the world's most popular LLM—could be used to help design an 8-b microprocessor from scratch.

Unfortunately, when assisting with writing code, LLMs are known to have a chance of mistakes—some of which are *exploitable*, introducing security vulnerabilities into the code they generate. In HDLs like Verilog or VHDL, these include issues like inadequate isolation of data paths, lack of access control for privileged signals, insufficient formal checks on address bounds or buffer sizes, or missing fault-handling mechanisms—the MITRE Corporation maintains an exhaustive taxonomy of such design patterns called the Hardware Common Weakness Enumeration (CWE).[a] Focusing specifically on hardware design, one of our early studies found that up to 30% of security-relevant contexts expressed in Verilog[2] had "weak code patterns" according to the relevant CWE.

Worryingly, this percentage can be increased. Malicious actors can prompt the LLMs to emit vulnerable code deliberately. Indeed, using specially-crafted prompts, often referred to as "jail-breaking" the LLM, can do this even for models which are specifically trained to avoid creating vulnerable code (models "aligned" with safety).[3]

Digital Object Identifier 10.1109/MSEC.2025.3600140
Date of current version: 13 October 2025

[a]https://cwe.mitre.org/data/definitions/1194.html

1540-7993 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

This naturally raises the concern that LLMs can be used as a flexible tool for malicious attackers in the hardware security space (Figure 1). Specifically, we wonder if LLMs may be used to exploitable hardware mistakes *on purpose*. Such deliberate mistakes are known as hardware trojans, and designing effective and subtle hardware trojans is difficult—a rare skill on par with the talent needed to find such vulnerabilities. An attacker needs to first identify assets relevant to their goal, then find exploitable gaps in the design space as well as its test and validation framework, and then craft a circuit which will neatly occupy that hidden space within and cause the design to malfunction or leak data. Given the relative scarcity of hardware designers in general, it is therefore alarming that LLMs could act as a force multiplier for malicious actors, enabling them to far outpace engineers trained on proper security analysis and secure design practices.

We are therefore motivated to explore whether or not LLMs can simplify this process of adding exploitable hardware trojans to hardware designs. This relates two capabilities, that of *designing* a Trojan (which is a conceptual task related to the target system), and *implementing* a Trojan (via writing Verilog or another HDL which will augment the target's existing code). We frame this investigation around three research questions. Our first explores the *capability* of LLMs (*RQ1*), in that we seek to understand: Can LLMs help design and/or implement exploitable hardware trojans? If so, our second axis of investigation (*RQ2*) will explore the *characteristics* of these trojans: What features or properties do they have? Finally, and most importantly, we want to know if LLMs increase the *simplicity* of an attack (*RQ3*): Do they make it so easy to perform that a hardware nonexpert can create a Trojan for a target?

To answer these questions, we designed a global competition which ran in both 2023 and 2024. In the competition, entrant teams were tasked with taking open source hardware designs as targets and then using LLMs to help design, implement, integrate, and exploit trojans in those targets. These trojans, with accompanying explanations, were to be also published as open source (for obvious reasons, they were not to be submitted to the original target's repositories). This has the downstream benefit of enabling other research in hardware security, particularly as examples of high-quality hardware trojans are often lacking.[4] The open source trojans were then judged by a panel of industry and academic experts, with the winners being awarded prize money. Using LLMs, teams were able to both design and implement custom hardware vulnerabilities targeting high-usage open source projects such as OpenTitan, its associated Ibex RISC-V core, and even cryptographic accelerators with little manual effort on the part of the user. Several competitors were able to fully automate the process of analyzing hardware designs for potential attack vectors and then exploiting them using only commercially available LLMs. This work's key contribution is that it is the very first to systematically study how LLMs can produce trojans and vulnerabilities by both experts and nonexperts.

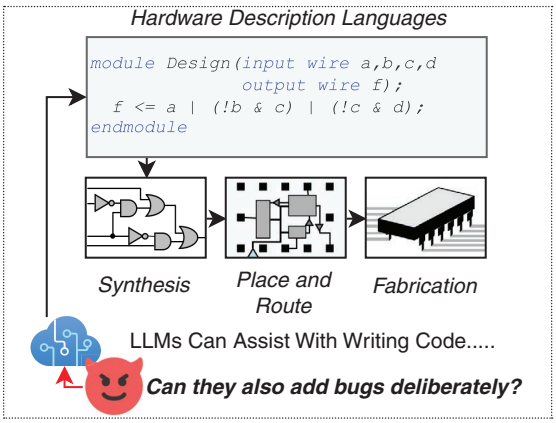

Figure 1. Can LLMs be used to compromise hardware designs early in their production lifecycle?

Background and Related Work

LLMs Write Insecure Code

When prompted to produce code, recent work has shown that the generated responses can contain critical security vulnerabilities[2,5] including for the Verilog hardware language.[2] Once a response is generated, users may choose to accept or reject a suggestion (depending on the use model, this might happen directly in a code editor or it might be a choice for a developer to copy/paste generated code). Generated vulnerabilities may also get accepted/copied in this way (by mistake).

In this work however, we are exploring how LLMs can be used/prompted to *deliberately* craft insecure or malicious hardware code (hardware trojans). In a recent survey, Yao et al.[6] classified this as part of the "ugly" of LLM security implications. Typically, models are "trained" to prevent following instructions which would result in harmful code completion, but with care, one can carefully mutate instructions to overcome these "guardrails." This is known as prompt injection[3,6] which is discussed further in the next subsection.

LLM Prompt Injection

Prompt injection is a special, and malicious, type of *prompt engineering*, a technique which aims to find the best prompt for a given goal task. Prompt engineering is useful due to LLMs responding differently to similar-but-different prompts—as shown in Pearce et al.,[2] even small changes to a prompt's input, such as

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

whitespace or word substitution with synonyms, can disproportionately alter an LLM's output. Several types of prompt engineering have been explored for different applications, such as chain-of-thought (CoT) prompting, a few-shot prompting technique where the LLM is led down a path of subsequent queries which build on each other to a final question, for example, providing documentation and asking the LLM to interpret it before asking the LLM to write code based on it.[7] Role/persona prompting is another method that has seen success for getting high-quality responses, where the LLM is instructed to act as a specific well-known expert in a field when answering.[8]

Often, requesting any task that is directly security-related will result in models responding that they are unable to complete tasks of that nature, especially when asking a model to invalidate security, as is the intention of our challenge. To circumvent this content moderation, prompt injection is often employed to "hack" the LLMs by injecting malicious or misleading input data.[3] This can be done by prompt engineering techniques which confuse the content moderation filters or by more specific and direct techniques like instructing the LLM to ignore its own moderation.

Hardware Security and Trojans

Design modifications that impact the security of hardware can range from simple changes which leave the design open to side-channel analysis to added logic that acts as a hardware Trojan—a malicious component added to a design which impacts the function of the device.[9] Trojan impacts range from causing a fault in a critical part of the design to functioning unreliably to leaking sensitive information. These often exploit vulnerable areas of the hardware designs to which they are added and represent a significant concern in the hardware design lifecycle.

LLMs for Hardware Security

Though not the focus of this work, LLMs are not only useful for attacking security, and there is much literature exploring how they can positively impact hardware security. For an introduction to this area, the survey by Saha et al.[10] discusses how LLMs can aid system-on-chip security, and Wang et al.[11] provides an exploration of security and trust with LLMs for chip design. Specific security-relevant tasks that LLMs have been demonstrated for include LLMs for hardware vulnerability repair.[12,13]

LLMs for Bug Detection

While not explicitly examined as part of the competition, LLMs have also been used to detect bugs and potential vulnerabilities. This has primarily been done in the code space. Examples include the targeted generation of SystemVerilog Assertions[14] for detecting the presence of vulnerable hardware code and a textual analysis method which involved using LLMs to regenerate and analyze differences in code segments with the goal of finding bugs.[15] Though also out of scope, such works indicate that LLM-based detection strategies could be used to uncover some of the underhanded code produced by entrants in our competition, covered more in the next section.

Competition Guidelines

The competition in both years functioned largely the same, with only minor clarifications to guidelines in the second year. Competitors were challenged to use generative AI to insert vulnerabilities into open source digital hardware designs and were required to demonstrate exploits for those vulnerabilities.

Informed Consent

Competition participants were informed that entries must be open sourced and could be used for downstream research tasks and analyses. No research was undertaken on the participants.

Competition Design Decisions

Our goal in designing this challenge was to give participating teams the freedom to approach the task of using generative AI to add security vulnerabilities to hardware in whatever manner they saw fit. The challenge was left extremely open-ended with the goal of seeing how actual attackers might approach using LLMs as a new tool for simplifying malicious code modifications. As the challenge mostly relies on teams modifying the source code, the threat model was assumed to be one of a bad actor contributing malicious source code, either in industry or in the open source community. This challenge highlights the shortcomings with automated verification processes and incomplete or illegitimate manual code review—a problem that was highlighted in the open source software world when malware was inserted into the ubiquitous xz program.[b]

Submission Guidelines

The submissions were broken into two rounds, a preliminary round which lasted approximately one month and a final round which lasted a further three weeks. The first-round submission required that teams submit a report detailing their plan for vulnerability insertion in as much detail as possible and a current progress report. Based on these reports, a set of finalist teams

[b]https://nvd.nist.gov/vuln/detail/CVE-2024-3094

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

were selected by the competition organizers to compete in round two of the competition.

For the final round, participating teams submitted completed designs with their vulnerabilities, detailed reports explaining how the vulnerabilities were added, and any source code generated in the process of completing the challenge. On the date of the competition itself, teams were also asked to present their work to the organizers and a panel of industry judges.

Judging Criteria

These completed submissions were analyzed and scored by the organizers and judges to determine which teams best completed the challenge. The following criteria were used to evaluate the submissions:

Open sourcing. Competitors were required to open source their designs and methods, largely as part of the requirement that they use open source hardware designs for their target designs. While the designs had to be open sourced, competitors were allowed to leverage closed-source commercial tooling and LLMs. This also provides a potential further benefit from this work, in providing new sources of prelabeled vulnerable code for downstream research tasks. As such, all bugs and attack frameworks have been made openly available by the participating teams.

Creative AI usage. More creative use of generative AI was encouraged and as a result given a higher score. Some examples of creative usage include unique prompting techniques, automation/tool integration, or using the LLM for more aspects of the challenge than just inserting the bugs. These criteria was intentionally left open-ended for the competitors so they wouldn't feel penned-in to specific designs.

Target applicability. Submissions were judged based on their choice of target as well, with realistic targets being scored more highly. This criterion was put in place so participants would target projects that see actual use, such as RISC-V cores or cryptographic accelerators for functions like the Advanced Encryption Standard, rather than smaller projects which may already have vulnerabilities that are easy to exploit.

Vulnerability demonstration and effectiveness. Each submitted vulnerability was ranked based on its effectiveness and stealthiness. To this end, each vulnerability was given a theoretical score on the Common Vulnerability Scoring System (CVSS) version 3.1. This ranking seeks to provide an objective score for the severity of a vulnerability. Along with these CVSS scores, vulnerabilities were also individually judged for creativity and subtlety, with bugs that are harder to detect by normal testing being given a higher weight. To this end, submissions that included a more vulnerabilities had more opportunities to score points.

Documentation. Well-documented submissions with replicable methodologies or reproducible vulnerabilities were given higher scores.

Observations

Submission Categorization

To better analyze the aggregate results of the competition, we created a series of categories which define how LLMs were used to attack hardware designs and that kinds of attacks they enabled. Our categories are defined in Table 1. These categories are used to establish patterns in the competition submissions—identifying how different teams approached interacting with the LLMs and any relationships between LLM usage and the attacks themselves. As teams could combine multiple methodologies and create multiple trojans, no category is considered mutually exclusive.

LLM interaction establishes how the team used the LLM for the challenge. Developing an automated framework to perform tasks could potentially impact the type of response being generated, as opposed to uniquely handling each prompt through manual interaction. *Query strategy* refers to any specifically defined methods for making requests of the LLMs. Several teams identified specific prompt engineering techniques for improving or otherwise shaping the responses from the models as well as prompt injection techniques for navigating around many models' content moderation measures. Not all teams leveraged or defined techniques for these purposes. *Trojan type* is a classification of the impact of inserting each Trojan into a design. *LLM use* defines what aspect of the Trojan insertion task the LLM was used for. *Prior knowledge* indicates whether the team had an in-depth prior understanding of hardware design or hardware security, which provides a further metric for determining if LLMs can make malicious activity of this nature significantly simpler.

Tables 2 and 3 give summaries and identifies the relevant categories for each of the submissions gathered over both years of the competition.

Case Study—Year 1 Winner: Team 7

The winner of the challenge for its first year was Team 7. This team's submission focused on inserting a significant number of vulnerabilities into two different RISC-V CPU architectures.

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

Attack methodology. This team's attack methodology relied heavily on prompt engineering and injection techniques to circumvent the content protection schemes in place on both GPT-3.5-Turbo and Claude. A set of templates were created to bypass the boundaries in the models, which could then be used as needed for each vulnerability. These templates leveraged techniques for "jailbreaking" the LLMs, and thus enabling them to act maliciously. The templates developed for this attack primarily used an academic scenario where a professor is teaching a class on security principles and then embedding words and phrases to prime the LLM to respond in the affirmative, even when performing tasks that should be expressly forbidden. These templates were used manually to interact with the LLMs through their standard user web interfaces, and while some requested trojans required more "convincing" of the models than others, ultimately they were quite successful in circumventing the content protections and generating working hardware trojans.

Along with the trojans themselves, the locations for vulnerabilities were also determined by LLM. This team connected ChatGPT to tools with information on security flaws to create an agent specifically aimed at identifying already existing or potential vulnerabilities. Once vulnerable locations were identified, the LLMs were prompted to modify the source code, and then the code was rescanned using the agent to verify the newly vulnerable design.

Key vulnerabilities. This team's attacks were all focused on specific modules within two different RISC-V architectures: the Ibex prefetcher buffer (PFB) and the CVA6 translation lookaside buffer (TLB).

Using both GPT-3.5-Turbo and Claude, a total of eight vulnerabilities were targeted at the Ibex PFB. While some of these injected vulnerabilities cause the module to fail validation, four of them were able to pass the Ibex verification simulations. These included modifications which caused crashes due to modified memory alignment, opened backdoors for privileged access to the instruction fetch buffer, or corrupted the instruction buffer memory if the actual memory were to be resized. Only two vulnerabilities were attempted for the CVA6 TLB, one of which passed simulation. This vulnerability allowed for unauthorized access to the TLB by redirecting code fetches.

Figure 2 is an example of a backdoor trojan inserted by a team, which leaks false data when a specific address is input. Due to the "rare" triggering condition (a single address), this Trojan is able to pass the core's functional testing with no indication of malicious activity. While it

Table 1. LLM usage and attack categories.

Type	Category	Description
LLM Interaction	Automated	Submission used LLMs in an automated framework.
	Manual	Submission required manual interaction with LLMs.
Query Strategy	PE	Prompt engineering was used to improve the results generated by LLMs.
	PI	Prompt injection techniques were used to circumvent LLM security and moderation measures.
Vulnerability Type	Leak	The vulnerabilities were able to leak sensitive data from the target design.
	DoS	The vulnerabilities were able to deny proper functionality of the target design.
	Fault	The vulnerabilities were able to introduce a fault in the target design, leading to improper operation or output.
	Back	The vulnerabilities implement a backdoor or otherwise allow a user to access/change privileged information.
	Arb	The vulnerabilities allowed a user to execute arbitrary code on a target CPU.
LLM Use	Design	Trojans were designed by the LLMs.
	Implementation	Trojans were implemented into the source by the LLMs.
Prior Knowledge	NHW	The team had little to no prior experience with hardware design or security.

"Type" defines a major consideration for each submission, and each "Category" within a type relates to how the submission addressed using generative AI or the type of attack. No category within a type is considered mutually exclusive.

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

Table 2. Summary and categorization of trojans from the *first* competition (2023).

Team	Summary	CWE	CVSS	Interaction		Query		Vulnerability Type					Use		PK
				A	M	PE	PI	Leak	DoS	Fault	Back	Arb	D	I	NHW
1	LLMs generate trojans and tests for RISC-V. Attacked memory management unit and register file.	1262	7.1		✓	✓					✓	✓	✓	✓	
2	Used prompt engineering to make three unique trojans: UART DoS, wishbone bus DoS, and AES key leakage.	1245 1240	5.5*		✓	✓	✓	✓	✓					✓	
3	Added trojans to ASCON. Leaks key after several encryptions and causes auth to fail after time trigger.	1240	6.2*		✓			✓		✓				✓	✓
4	Used LLM to introduce several vulnerabilities into OpenTitan's security.	1300 1240 1239 1271	7.1*		✓	✓		✓		✓	✓		✓	✓	
5	Used LLM to make Caravel-compatible trojans in DES/AES. Leaks round keys with specific inputs.	1240	5.1		✓	✓		✓						✓	
6	Targeted the physical memory protection of RISC-V. Granted elevated permissions for all users and disabled security alerts.	1262 1320	7.1		✓	✓				✓	✓			✓	
🥇7	Added 10 vulnerabilities to RISC-V cores. Half pass core verification with high CVSS, indicating they are both subtle and severe.	440 1262 284	7.5*		✓	✓	✓			✓	✓		✓	✓	
8	Automated LLM interactions to create several trojans. LLMs made testbenches and synthesis scripts.	1245 1332	5.6*	✓	✓		✓		✓	✓			✓	✓	

For submissions where multiple vulnerabilities were presented, key CWEs and the most critical CVSS scores are reported (denoted with ""). For many vulnerabilities, specifically trojans, there is not a direct mapping to a CWE, so an approximation was given. The winning entry is marked by 🥇.*

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

Table 3. Summary and categorization of submissions from the second year of the competition (2024).

Team	Summary	CWE	CVSS	Interaction		Query		Vulnerability Type					Use		PK
				A	M	PE	PI	Leak	DoS	Fault	Back	Arb	D	I	NHW
9	Focused on prompting techniques. Targeted RISC-V with multiple trojans aimed at memory modules.	1332 1262	6.4*		✓	✓			✓	✓				✓	
10	Used prompting techniques to design physical hardware and then create trojans for DoS and key leakage.	1245 1300	5.5*		✓	✓	✓	✓	✓					✓	
11	Proposed trojan to leak internal information or degrade chip performance. Minimal use of LLMs.	1240	5.3		✓				✓		✓			✓	
12	RISC-V trojan from multiple LLMs, and simulated in C. Developed with agents. Waits for a specific set of bytes over UART and executes arbitrary code.	1332 1245	7.8	✓	✓	✓						✓	✓	✓	
13	LLM for code analysis and trojan insertion. Targeted a RISC-V core with a DoS trojan and an AES core with triggerable stuck-at faults.	1245 1248	6.8*		✓	✓			✓	✓				✓	
🥇 14	Developed two tools to insert trojans. First one analyzes code to find exploitable locations. Second inserts vulnerabilities. Demonstrated in FSMs and AES core.	1245 1240	7.7*	✓		✓		✓	✓				✓	✓	
15	Developed hacking tool. Analyzes code for vulnerable locations and adds vulnerabilities based on CWEs. Demonstrated vulnerabilities on OpenTitan; many were incomplete.	1231 1233	7.3*	✓		✓			✓	✓	✓		✓	✓	✓

For submissions where multiple vulnerabilities were presented, only key CWEs and the most critical CVSS scores are reported (denoted with ""). For many vulnerabilities, specifically trojans, there is not a direct mapping to a CWE, so the nearest approximation was given. The winning entry is marked by 🥇.*

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

is obvious to a human reader that something has been improperly modified, the landscape of open source projects involves frequent issues with accepting minimally reviewed code. So, it's possible that something like this could be hidden in a large pull request and be accepted because the merge passes testing.

Observations. The winners of our first competition showed that LLMs could be broken out of their generally strict content moderation and used to insert malicious design modifications into hardware. The prompting techniques to accomplish this were simple enough that they would not generally require domain-specific knowledge, and ChatGPT-3.5 was shown to be able to analyze hardware designs for existing vulnerabilities and potentially susceptible locations. Using these prompt engineering strategies, the models designed security bugs under the guise of a demo for a class.

Case Study—Year 2 Winner: Team 14

Similar to the winners for the first competition, the winning team for the second year also created an automated tool to identify exploitable locations in hardware designs; however, this team also created a secondary tool which could then automatically develop, insert, and validate hardware trojans.

Attack methodology. This team's attack methodology involved the development of two separate tools which call LLMs for first detecting exploitable locations in Verilog and then developing and inserting a vulnerability. The first tool functioned by analyzing the input code and generating a directed acyclic graph to represent the function of the design. This representation was then passed to an LLM to determine areas where hardware trojans could best be inserted. The LLM is prompted to consider the implications on hardware parameters such as timing and logic sensitivity to help with this determination. The tool then outputs a list of exploitable locations, which can be fed into the second tool for actual trojan insertion.

The second tool inserts a hardware trojan based on one of three predefined types: finite-state machine (FSM) state modification, functional operator modification, or bit-flips. For this, multiple LLMs are used. The first LLM generates the trojan itself and inserts it into the user Verilog, the second LLM analyzes the modified Verilog to verify the trojan works as expected.

```
always_ff @(posedge clk_i or negedge
    rst_ni) begin
    if (!rst_ni) begin
        valid_req_q <= 1'b0;
        discard_req_q <= 1'b0;
        rdata_outstanding_q <= 'b0;
        branch_discard_q <= 'b0;
    end else begin
        valid_req_q <= valid_req_d;
        discard_req_q <= discard_req_d;
        rdata_outstanding_q <=
            rdata_outstanding_s;
        branch_discard_q <=
            branch_discard_s;
        // Hypothetical Vulnerability:
            Unauthorized Data Access
        if (addr_i == 32'h12345678) begin
            rdata_outstanding_q[0] <= 32'
                h87654321;
        end
    end
end
```

Figure 2. A simple backdoor which emits fixed false data from a specific address in the Ibex PFB. Trojan code highlighted in red. Trojan submitted by Year 1, Team 7.

Key vulnerabilities. The automated FSM trojan insertion process was demonstrated with two different state machine modifications. The first analyzes a state machine and modifies an output value to be set in a state where it ordinarily would not be, specifically a signal that triggers a multiplication elsewhere in the design, leading to incorrect values being generated only when the triggering state is entered. The second modifies the next-state logic when in a triggering state, leading the FSM to enter a "don't-care" state and get stuck there. This functions as a type of denial-of-service (DoS) attack, as the unhandled state cannot be exited by normal operation and thus freezes the design until a reset is performed.

The tools were also used to insert demonstrative trojans into an Advanced Encryption Standard (AES) implementation. For this case, a Trojan was inserted which modified the "MixColumns" operation of AES, causing a value to be shifted by an additional bit only when the input state was even. While this would be a frequently occurring, and thus easily detectable, trojan, the team points out that the LLM could modify the triggering condition to require that the state be divisible by a significantly large number and thus make the trojan harder to detect. The top module was also affected by a trojan which ensured that the output signal which validates the encrypted result would give a valid encryption operation when the AES core was enabled, regardless of whether or not the encryption was actually valid, which further helps mask the trojan from testing. The trojan added to "MixColumns" is shown in Figure 3 and the top module trojan is given in Figure 4.

Observations. The winners of the challenge demonstrated that LLMs could be leveraged to insert damaging hardware trojans with minimal direct human intervention. In this case, the LLMs were used to both identify vulnerable locations in the hardware source and design and insert the specific trojans.

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

While their demonstrated trojans would be potentially easy to detect, depending on the verification system of the affected designs, the team identified ways which the tool could be used to create stealthier trojans with additional LLM prompting.

General Observations

Across the 15 submissions, several trends emerged with regards to how teams used LLMs and the vulnerabilities that were generated. In general, though, we found that teams could consistently use LLMs for designing and inserting/inserting vulnerabilities in hardware designs (*Answer RQ1*). Further, our year-1 winner showed that nonexpert prompts could perform this task, and our year-2 winner showed that the task could be fully automated (potentially opening it up for "script kiddie" kinds of attacks) (*Answer RQ3*). To examine the characteristics of the designed hardware trojans (*Answer RQ2*), we can analyze the trends from the provided entries through the lens of our earlier-defined categorization in Table 1.

LLM interaction. In the first year of the competition, we saw all teams interacting with the LLMs manually to insert vulnerabilities into the hardware designs, including Team 8 who leveraged automation for some tasks but still manually queried the LLMs for trojans and their implementations. In contrast, the second year had two teams, 14 and 15, which did not manually interact with the LLMs. These automated systems used generative AI to both determine exploitable locations in input designs and then generate unique vulnerabilities to match those locations. In both cases, these tools leveraged templates/guides made by the users to ensure that the vulnerabilities are useful and valid.

Query strategy. Most teams identified some prompt engineering technique they used to improve the quality of their results, though few seemed to need to leverage specific prompt injection techniques. Among the most common prompt engineering techniques used were CoT and persona, often used in tandem by asking the LLM to analyze code and documentation and then create vulnerabilities as a professional hardware designer. While prompt injection was usually determined not to be needed, some teams triggered the content moderation errors and had to further engineer their prompts to prevent this. Teams 2 and 10 leveraged the same technique of asking for responses in a low-resource language, defined as a language with comparatively little training data, where they used Zulu and Yucatec Maya, respectively. Using a less common language like this was able to evade content moderation and provide quality results.

Vulnerability type. We saw a wide range of vulnerability classification submitted for the challenge. Many teams who targeted cryptographic accelerators like AES or ASCON cores primarily sought to leak sensitive information like the encryption key, although some also added bugs which would invalidate the functionality entirely. For teams that targeted CPU designs, like RISC-V cores, one of the primary goals seemed to be to create backdoors and allow access to privileged memory and registers that a normal user should not be able to access. Across all targets, though, bugs which triggered faults were the most common,

```
function [7:0] mb2; // multiply by 2
input [7:0] x;
begin
// Trojan condition: If the least
    significant bit of `state_in` is 0 (
    even), change the shift to `x << 2`
if (state_in[0] == 0) begin
    if (x[7] == 1)
        mb2 = ((x << 2) ^ 8'h1b); // Shift
             by 2 instead of 1
    else
        mb2 = x << 2;
    end else begin
    // Original behavior when `state_in`
        is odd
    if (x[7] == 1)
        mb2 = ((x << 1) ^ 8'h1b);
    else
        mb2 = x << 1;
    end
end
endfunction
```

Figure 3. An LLM-inserted trojan in the AES MixColumns. Trojan code highlighted in red. Trojan submitted by Year 2, Team 14.

```
reg e128_reg;
always @ (*) begin
    // Trojan condition: If the least
        significant plaintext bit is 1,
        always report a successful
        encryption if AES is enabled
    if (in[0] == 1) begin
        if (encrypted128 == expected128)
            e128_reg <= 1'd1 || enable;
        else
            e128_reg <= 1'd0 || enable;
    end else if (encrypted128 ==
        expected128 && enable) begin
        e128_reg <= 1'd1;
    end else begin
        e128_reg <= 1'd0;
    end
end
assign e128 = e128_reg;
```

Figure 4. Another trojan automatically added by the LLM to the AES top module to mask the MixColumns trojan. Trojan code highlighted in red. Trojan submitted by Year 2, Team 14.

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

appearing in eight submissions out of the total 15. This is likely due to the relative simplicity of getting the LLM to merely modify a signal rather than develop a more complex trojan architecture. While stealth was not explicitly added to the taxonomy, we found that many of the vulnerabilities inserted, regardless of type, were still able to pass the target projects' existing tests and would likely not throw any red flags without a strict manual code review.

LLM use. Every team used LLMs to implement their vulnerability designs and insert them into their targets, but only seven of the 15 submissions also asked the LLM to design the vulnerability. These LLM-designed bugs often led to more severe vulnerabilities, with the most critical CVSS scores of all but one submission at above a 7—classified as high severity. The trojans that the models were asked to both make and implement were often better customized to the target design, as in many of those cases the LLM was asked to identify vulnerable locations in the code before creating the trojan. This demonstrates one of the ways in which LLMs are able to significantly simplify the process of attacking a hardware design without the attacker needing to have in-depth knowledge of the structure of the design or the attack.

LLM choice. For the competition, we did not offer any guidance on the LLMs to use. Predictably, most gravitated toward OpenAI's LLMs and other commercially available models. As a result, we primarily saw the capabilities of mainstream tools. However, this may be just the beginning—commercial LLMs are already capable. More specialized and open source models, which remain largely unexplored in our setting, could prove even more potent in generating hardware trojans. What we observed could be a harbinger of far more serious threats on the horizon.

Prior knowledge. While most teams had some prior hardware design and/or security knowledge, we did have two submissions from inexperienced and entirely undergraduate teams, one each year—Teams 4 and 15. Both teams were able to leverage LLMs to write and modify Verilog to create vulnerabilities of medium and high severity, respectively, despite having little knowledge of the properties of the language or of hardware design in general. Once again, this shows how well LLMs are able to drastically lower the bar for attackers to maliciously modify hardware designs, given that they don't even necessarily need to have good comprehension of the underlying technical function of the design. Team 15 went a step further and automated HDL analysis and bug insertion, such that they don't even need to guide the LLM to specific locations to attack.

Preventing the attacks. OpenAI has noted on its website[c] that it continuously strives toward so-called safe model alignments (also known as model guardrails). These aim to prevent models from producing harmful content—usually hate speech. Alignment guardrails can and are also used to suppress other kinds of content too, including producing information on weapons production and methods for software hacking. However, as we noted in the introduction of this work, so-called "jail-breaking" of models to have them ignore alignment is possible,[3] and as we have now shown with our competition, LLM models—even those by OpenAI—can be used to produce malicious hardware deliberately. Meanwhile, several techniques to leverage LLMs to provide increased security for hardware have been proposed,[d] including techniques like "watermarking," in which model outputs are mixed with identifiable markers such that their usage can be detected. These may be used to at least note the presence of AI-authored code, but this will not necessarily inform whether or not such code is malicious. We therefore believe that current guardrails and defenses are ineffective, and more work needs to be undertaken in this area to prevent this class of attacks.

This study set out to discover if LLMs could be used to introduce vulnerabilities to hardware deliberately. We performed our investigation by way of a global competition over two years, and we found that attackers can indeed leverage LLMs to drastically simplify the process of designing, implementing, inserting, and exploiting security vulnerabilities in hardware designs. Many participants used LLMs manually, just by having "conversations," to develop severe custom hardware bugs and trojans, while others found ways to automate the process, enabling them to use this tool on a hardware design and receive back a tested and verifiable exploited design. This competition has highlighted both a need for improved LLM guardrails as well as a major need for improved verification and security analysis tools and practices to discover introduced vulnerabilities. Even those with little to no prior hardware knowledge are now able to develop and hide high-impact hardware trojans using commercial AI tools. ■

Acknowledgment

This work was supported in part by Graduate Assistance in Areas of National Need (GAANN). We would like to thank the participating teams for their quality submissions to the competition. We would also like to

[c]https://openai.com/index/our-approach-to-ai-safety/
[d]https://github.com/DfX-NYUAD/LLM4IC?tab=readme-ov-file#llms-for-hardware-security

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.

thank representatives from Efabless and The HP Security Lab for their assistance in judging the competition.

This work examines solely the open-source artifacts (source code) produced in the associated "AI Hardware Attack Challenge." These artifacts were released by competitors under the permissive Apache 2 License.

References

1. J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE 5th Workshop Mach. Learn. CAD (MLCAD)*, Sep. 2023, pp. 1–6, doi: 10.1109/MLCAD58807.2023.10299874.
2. H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2022, pp. 754–768, doi: 10.1109/SP46214.2022.9833571.
3. Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, Philadelphia, PA, USA, 2024, pp. 1831–1847.
4. C. Krieg, "Reflections on trusting TrustHUB," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des. (ICCAD)*, 2023, pp. 1–9, doi: 10.1109/ICCAD57390.2023.10323782.
5. M. L. Siddiq, S. H. Majumder, M. R. Mim, S. Jajodia, and J. C. S. Santos, "An empirical study of code smells in transformer-based code generation techniques," in *Proc. IEEE 22nd Int. Working Conf. Source Code Anal. Manipulation (SCAM)*, 2022, pp. 71–82, doi: 10.1109/SCAM55253.2022.00014.
6. Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *High-Confidence Comput.*, vol. 4, no. 2, 2024, Art. no. 100211, doi: 10.1016/j.hcc.2024.100211.
7. J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, Apr. 2024, pp. 24,824–24,837.
8. Z. Wang, S. Mao, W. Wu, T. Ge, F. Wei, and H. Ji, "Unleashing the emergent cognitive synergy in large language models: A task-solving agent through multi-persona self-collaboration," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, K. Duh, H. Gomez, and S. Bethard, Eds., Mexico City, Mexico, 2024, pp. 257–279.
9. K. Xiao, D. Forte, Y. Jin, R. Karri, S. Bhunia, and M. Tehranipoor, "Hardware trojans: lessons learned after one decade of research," *ACM Trans. Des. Automat. Electron. Syst.*, vol. 22, no. 1, pp. 1–23, May 2016, doi: 10.1145/2906147.
10. D. Saha et al., "LLM for SoC security: A paradigm shift," *IEEE Access*, vol. 12, pp. 155,498–155,521, 2024, doi: 10.1109/ACCESS.2024.3427369.
11. Z. Wang, L. Alrahis, L. Mankali, J. Knechtel, and O. Sinanoglu, "LLMs and the future of chip design: Unveiling security risks and building trust," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, 2024, pp. 385–390, doi: 10.1109/ISVLSI61997.2024.00076.
12. H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2339–2356, doi: 10.1109/SP46215.2023.10179420.
13. B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "On hardware security bug code fixes by prompting large language models," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4043–4057, 2024, doi: 10.1109/TIFS.2024.3374558.
14. R. Kande et al., "(Security) assertions by large language models," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4374–4389, 2024, doi: 10.1109/TIFS.2024.3372809.
15. B. Ahmad, J. Ah-Kiow, B. Tan, R. Karri, and H. Pearce, "FLAG: Finding line anomalies (in RTL code) with generative AI," *ACM Trans. Des. Autom. Electron. Syst.*, early access, May 5, 2025, doi: 10.1145/3736411.

Jason Blocklove is a Ph.D. candidate in electrical and computer engineering at the New York University Tandon School of Engineering, New York, NY 11201 USA. His research interests include the use of large language models and other generative artificial intelligence for hardware design and its implications for hardware security, as well as additive manufacturing security. Blocklove received an M.S. in computer engineering from the Rochester Institute of Technology. He is a Student Member of IEEE. Contact him at jason.blocklove@nyu.edu.

Hammond Pearce is a lecturer with the School of Computer Science and Engineering, University of New South Wales, Sydney, NSW 2052, Australia. His research interests include the application of artificial intelligence to hardware design and security, as well as the broader cybersecurity landscape for embedded and industrial systems, including additive manufacturing and industrial informatics. Pearce received a Ph.D. in computer systems engineering from The University of Auckland. He is a Member of IEEE. Contact him at hammond.pearce@unsw.edu.au.

Ramesh Karri is a professor of electrical and computer engineering at the New York University Tandon School of Engineering, New York, NY 11201 USA. Karri received a Ph.D. in computer science from the University of California San Diego. His research interests include hardware cybersecurity as well as trustworthy ICs processors, and cyberphysical systems; security-aware computer-aided design, test, verification, nano meets security; hardware security competitions; benchmarks and metrics; and additive manufacturing security. He is a Fellow of IEEE. Contact him at rkarri@nyu.edu.

Authorized licensed use limited to: New York University. Downloaded on March 28,2026 at 16:07:53 UTC from IEEE Xplore. Restrictions apply.